Deep Learning Module: Comprehensive MCQ-Friendly Cheat Sheet

Overview

This document covers all sessions from the **Generative AI: Transformers** module and earlier Deep Learning topics, organized for quick MCQ reference with key analogies and exam-ready patterns.

Session 25–27: Transformers Fundamentals

What Are Transformers?

- **Core mechanism**: Self-attention (parallel processing, not recurrence)
- **Key advantage over RNNs/LSTMs**:
  - Handle long-range dependencies directly (any word can attend to any other)
  - Process all tokens in parallel → much faster training
  - Better performance on large-scale NLP tasks
- **Architecture types**:
  - **Encoder-only**: BERT (bidirectional, text understanding)
  - **Decoder-only**: GPT (left-to-right generation)
  - **Encoder–decoder**: Original Transformer, T5 (sequence-to-sequence tasks)

Brief NLP Timeline

Era	Model Type	Key Limitation
Pre-2014	Rule-based, n-grams, HMMs	No deep learning
2014–2016	RNNs/LSTMs with attention	Slow, vanishing gradients
2017	"Attention Is All You Need" (Transformer)	None major; pure attention
2018+	Large pre-trained models (BERT, GPT)	Hallucination, bias, compute

Key insight: Transformers replaced RNNs because they process tokens in parallel and use attention to handle context globally, not sequentially.

Session 25–26: BERT (Bidirectional Encoder Representations from Transformers)

BERT Overview

- **Architecture**: Encoder-only transformer (reads text bidirectionally)
- **Pre-training approach**: Two-task approach on massive unlabeled corpora (Wikipedia, etc.)
- **Pre-trained once**, then fine-tuned many times for different downstream tasks

BERT Pre-training Objectives

Masked Language Modeling (MLM)

- Randomly mask ~15% of tokens in input
- Task: Predict masked tokens using both left and right context
- Makes BERT **bidirectional** (unlike older left-to-right LMs)
- Example: "The capital of India is [MASK]" → BERT predicts "Delhi"

Next Sentence Prediction (NSP)

- Input: Two sentences A and B
- Task: Predict whether B follows A in original corpus or is random
- Labels: "IsNext" or "NotNext"
- Helps model understand sentence-level relationships (useful for QA, entailment)

BERT Fine-tuning Recipe

1. Take pre-trained BERT weights (from MLM + NSP)
2. Add small task-specific head (classification layer, span predictor, etc.)
3. Train on labeled task data; BERT weights adapt slightly
4. Freeze some layers or adjust learning rate as needed

BERT Applications (Examples)

Sentiment Analysis

- Input: Review text
- Add classification head on [CLS] token (special classification token)
- Output: Positive/Negative/Neutral label
- [CLS] embedding = summary of entire review → simple classifier

Question Answering

- Input format: "[CLS] Question [SEP] Passage [SEP]"
- Task: Predict start and end positions of answer span in passage
- Two small classifiers: one for span start, one for span end
- Dataset example: SQuAD

Session 25–26: Text Preprocessing & Representation

Text Preprocessing Techniques

Technique	Definition	Example	Trade-off
Tokenization	Split text into words/subwords/characters	"Cats are running" → ["Cats", "are", "running"]	Subword tokens fit vocabulary better
Stemming	Chop suffixes to get rough root form	"playing", "played" → "play"	Fast but creates non-dictionary words ("hous" from "houses")
Lemmatization	Reduce to dictionary base form using grammar	"running", "ran" → "run"; "better" → "good"	Accurate but slower; requires POS tagging

Key MCQ distinction: Stemming is rough and fast (removes suffixes blindly); lemmatization is accurate and uses grammar knowledge.

Text Representation for Machine Learning

Bag of Words (BoW)

- Represent document as vector of word counts over fixed vocabulary
- Ignores word order completely
- Example: vocab = ["cat", "dog", "runs"]; "cat runs" → [1, 0, 1]
- Use case: Simple classification tasks

TF-IDF (Term Frequency–Inverse Document Frequency)

- Weight each word by how common it is in document vs entire corpus
- Common words ("the", "is") get low weight; rare discriminative words get high weight
- Formula idea: Important words are frequent in one doc but rare overall
- Better than BoW for classification

Word Embeddings (e.g., Word2Vec, GloVe)

- Map words to dense vectors (e.g., 300-dimensional)
- Similar words → similar vectors
- Static: same vector regardless of context

Contextual Embeddings (BERT, etc.)

- Word's vector depends on surrounding context

- "bank" in "river bank" ≠ "bank" in "savings bank"
- Learned during pre-training and fine-tuning

Session 28: Encoder–Decoder Architecture & Attention

Encoder–Decoder Basics

- **Encoder**: Reads full input sequence → produces hidden state representations capturing meaning
- **Decoder**: Generates output sequence one token at a time, using encoder outputs + previous tokens
- **Typical flow**: Input (e.g., English) → Encoder → Hidden states → Decoder → Output (e.g., Hindi)

Tasks Using Encoder–Decoder

- **Machine translation**: Source language → Target language
- **Text summarization**: Long document → Short summary
- **Question answering**: Question + context → Answer
- **Image captioning**: Image → Text description

Self-Attention Mechanism

What is attention?

- Mechanism that weights different parts of input differently when producing output
- Instead of compressing everything into one fixed vector, compute weighted sum over input states
- Weights indicate relevance/importance of each input element

Self-attention (in encoder)

- Each position attends to all other positions in the **same** sequence
- Creates a weighted sum of input representations
- Makes encoder bidirectional: each token sees both left and right context
- Key components: Query (Q), Key (K), Value (V) all from same sequence

Decoder self-attention

- Each position attends only to current and previously generated tokens
- Cannot see future tokens (preserves auto-regressive property)
- Prevents information leakage from future positions

Masked Attention (Decoder)

- **Why masked**: Decoder generates left-to-right; cannot peek at future outputs
- **How**: Set attention weights to future positions to negative infinity (or mask before softmax)
- **Mental picture**: Student writing an essay cannot look at words they haven't written yet

Attention's Impact

- **Solves fixed bottleneck problem**: Old encoder–decoder models compressed all context into one vector; attention lets decoder dynamically focus on different parts
- **Soft alignment**: For each output word, creates alignment weights across input words (learned and differentiable, unlike classical word alignment tables)
- **Long-range dependencies**: Any output token can attend to any input token, enabling handling of long sequences

---

Sessions 29–30: Large Language Models (LLMs) & Alignment

Introduction to LLMs

Definition

- Transformer-based model with **billions to hundreds of billions of parameters**
- Trained on massive text corpora (web, books, etc.) using language modeling objective: predict next token
- Can perform diverse NLP tasks with minimal task-specific training

Core Training Objective

- **Language modeling**: Given a sequence of tokens, predict the next token
- Trained on unlabeled, diverse internet-scale text
- No task-specific labels; learns general linguistic patterns

LLM Capabilities

- Text generation and completion
- Summarization and paraphrasing
- Machine translation
- Question answering
- Code generation and debugging
- Few-shot and zero-shot learning (learn from prompts, not training data)
- Information extraction and classification
- Creative writing and content generation

LLM Limitations (Critical for MCQs)

Limitation	Explanation	Example
Hallucination	Generates fluent but factually wrong/fabricated content	Citing non-existent papers confidently
Bias & stereotypes	Learns toxic, biased patterns from training data	Generating discriminatory text without prompting
Lack of grounding	Models text patterns, not real-world facts or causality	Cannot verify claims against external knowledge
Prompt sensitivity	Behavior changes significantly with minor prompt changes	"Jailbreaks" can bypass safety measures via adversarial prompts
No true understanding	Cannot reason about causality, verify facts, or self-correct reliably	Confidently provides wrong reasoning
Fundamental alignment limits	Even aligned models have non-zero probability of harmful behavior	Adversarial prompts can still trigger harmful outputs

Key exam insight: LLMs are powerful but fundamentally limited; no alignment method is perfectly safe.

Sessions 29–30: Alignment & Safety Strategies

Responsible LLM Development

Core Principles: Helpful, Harmless, Honest (with fairness, privacy, legality)

Practices:

- **Data curation**: Filter training data for toxic, illegal, highly biased content
- **Safety policies & system layers**: System prompts, safety classifiers, content filters to prevent harmful outputs
- **Transparency**: Clearly communicate model limitations; log and audit interactions
- **Privacy protection**: Avoid training on sensitive personal data; limit memorization
- **Continuous monitoring**: Regular evaluation against safety and fairness benchmarks; update policies as issues emerge

Reward Models & RLHF Pipeline

Standard Alignment Flow:

1. **Pre-train** large LLM on diverse internet text (language modeling objective)
2. **Supervised Fine-Tuning (SFT)**: Train on high-quality instruction–response pairs (e.g., "User asks X, helpful response is Y")
3. **Collect human preferences**: Annotators rank multiple model outputs for same prompt
4. **Train reward model**:
    - Smaller model trained on preference data
    - Learns to score outputs: higher score = more aligned with human preferences
    - Used as critic during next step
5. **RLHF (Reinforcement Learning from Human Feedback)**:
    - Use reward model as reward signal
    - Update main LLM to maximize reward model scores
    - Iteratively improve alignment

Alternative methods:

- **DPO (Direct Preference Optimization)**: Directly optimize on preference data without explicit RL loop
- **Constitutional AI**: Use written rules ("constitution") and other models to generate training signals

Aligning Goals with Human Values

Multi-objective alignment (not single metric):

- Helpfulness: Answer questions accurately, explain clearly
- Harmlessness: Avoid toxic, dangerous, illegal content
- Honesty: Admit uncertainty, avoid hallucinations
- Fairness: Treat all groups equitably; avoid discrimination
- Privacy: Respect user and third-party privacy

Techniques:

- **Reward design**: Encode human values in reward model (penalize hallucinations, toxic outputs, unfairness)
- **Continuous oversight**: Regular audits; update RM and model as new issues appear
- **Defense-in-depth**: Alignment alone insufficient; combine with safety filters and red-teaming

Mitigating Bias & Promoting Safety

Data-level mitigations:

- Audit training data for biased, toxic, stereotyped content
- Balance dataset to represent diverse groups and perspectives
- Remove or relabel harmful examples

Model-level mitigations:

- Debiasing during pre-training/fine-tuning (regularizers, contrastive objectives)

- Train reward models that penalize biased/unsafe outputs
- Address specific issues (e.g., length bias in reward models; longer responses sometimes scored higher unfairly)

System-level mitigations:

- Safety classifiers on outputs to filter harmful content
- Red-teaming: systematically probe model with adversarial prompts
- Logging and auditing interactions for suspicious patterns
- Human review of edge cases

Key MCQ point: Safety requires multiple layers; no single layer is sufficient.

Sessions 29–30: Real-World Applications

Chatbots & Virtual Assistants

- Customer support (answer FAQs, handle complaints)
- Tutoring and educational assistance
- Conversational search and information lookup
- Personal productivity assistants

Text Generation & Summarization

- Email and report drafting
- Marketing copy and content generation
- Document summarization (news, research, legal)
- Paraphrasing and style transfer

Code-Related Tasks

- Code completion and generation
- Code explanation and documentation
- Debugging and refactoring suggestions
- Learning programming languages

Other Applications

- Document question answering and search
- Machine translation
- Information extraction from unstructured text
- Text classification and sentiment analysis
- Data extraction and parsing

Sessions 29–30: Deployment & Infrastructure

Infrastructure Requirements

Hardware:

- **GPUs or TPUs** (almost mandatory for reasonable latency)
- Specialized accelerators (NVIDIA A100, etc.)
- High RAM/GPU memory to store model weights and activations
- Model parallelism across devices for very large models

Why GPU?

- LLMs have billions of parameters; CPU inference is impractically slow
- GPU parallelism essential for acceptable latency (sub-second queries)
- Extremely memory-intensive; often cannot fit on single GPU (multiple GPUs or model sharding)

Challenges

Challenge	Impact
High compute cost	Expensive GPUs, electricity, cooling
Memory bottleneck	Cannot fit full model + batch on single device
Latency requirements	Users expect fast responses; quantization/distillation may be needed
Variable traffic	Need autoscaling to handle demand spikes
Model updates	Retraining/fine-tuning takes significant compute

Efficient Deployment Strategies

Model compression:

- **Quantization**: Reduce precision (32-bit float → 8-bit or 4-bit); reduces memory and speeds up inference
- **Pruning**: Remove less important weights/neurons; trade-off between size and accuracy
- **Distillation**: Train smaller "student" model to mimic larger "teacher" model; much faster inference

Serving optimization:

- **Batching**: Combine multiple user requests into single batch for inference; improves GPU utilization
- **Caching**: Cache frequent prompts/responses to avoid redundant computation
- **Right-sizing**: Use smaller models for simple tasks; reserve large models for complex reasoning

Operational practices:

- Monitor latency, throughput, error rates, safety incidents
- Rolling updates and A/B testing when changing models or prompts
- Cost tracking and optimization (optimize $/inference)

Sessions 20–22: Regularization & Optimization

Regularization Techniques

Early stopping

- Monitor validation loss; stop training when it plateaus or increases
- Prevents overfitting and saves training time

Dropout

- Randomly drop units (set to 0) with probability p during training
- Forces network to learn robust, redundant features
- Not used during inference
- Prevents co-adaptation of neurons

L1/L2 regularization

- Add penalty term to loss: $L_{total} = L_{task} + \lambda \cdot R(w)$
  - L1: $R(w) = \sum |w_i|$ (sparsity; some weights go to 0)
  - L2: $R(w) = \sum w_i^2$ (distributed penalty; weights smaller overall)
- Constrains model complexity

Batch vs mini-batch vs stochastic training

- **Batch GD**: Update once per full dataset; stable but slow
- **SGD**: Update per single example; fast but noisy
- **Mini-batch GD**: Update per batch (32–256 samples); balanced, most common

Optimizers

Optimizer	Key idea	Pros	Cons
Batch GD	Update once per epoch	Stable updates	Very slow
SGD (stochastic)	Update per sample	Fast, noisy escapes local minima	High variance, noisy
Mini-batch GD	Update per batch	Balanced efficiency and stability	Hyperparameter tuning needed
Momentum	Accumulate gradient direction (like rolling ball)	Escapes plateaus faster	Overshooting possible
Adam	Adaptive learning rate + momentum (combines benefits)	Very popular, robust, fast convergence	Sometimes generalizes worse than SGD

Backpropagation of errors: Chain rule for computing gradients layer by layer; enables training deep networks.

Sessions 18–19: Neural Network Fundamentals

Single-Layer Perceptron

- **Structure**: Input layer directly connected to output neuron
- **Capability**: Solves only linearly separable problems (e.g., AND, OR)
- **Limitation**: Cannot solve XOR or other non-linearly separable problems
- **Learning rule**: Perceptron learning rule; update weights if misclassified

Multi-Layer Networks

- **Add hidden layers** to learn non-linear patterns
- **Universal approximation**: With enough hidden neurons, can approximate any function
- **Backpropagation**: Compute gradients using chain rule; enable training of deep networks

Activation Functions

Function	Formula	Range	Properties
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	(0, 1)	Smooth, historical; vanishing gradient problem
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1, 1)	Symmetric around 0; better than sigmoid
ReLU	$\max(0, x)$	[0, ∞)	Simple, fast, popular; dying ReLU problem possible

Modern practice: ReLU and variants (LeakyReLU, ELU) dominate; rarely use sigmoid/tanh in hidden layers.

Autoencoders

- **Structure**: Encoder (compress) → bottleneck → Decoder (reconstruct)
- **Goal**: Learn compact meaningful representation without labels
- **Objective**: Minimize reconstruction loss $L = ||x - \hat{x}||^2$
- **Applications**: Dimensionality reduction, anomaly detection, feature learning
- **Intuition**: Force network to learn low-dimensional representation that preserves important info

---

Sessions 17–19: Deep Learning Essentials & Frameworks

Deep Learning Basics

- **Neural network**: Layers of neurons with learnable weights and biases
- **Deep**: Multiple hidden layers (3+)
- **Supervised learning**: Learn from labeled data
- **Training**: Minimize loss function via backpropagation and gradient descent

TensorFlow & Keras

- **TensorFlow**: Open-source framework by Google; production-friendly
- **Keras**: High-level API (part of TensorFlow); easier to use
- **Core concepts**:
  - **Tensors**: N-dimensional arrays (like NumPy arrays but with GPU support)
  - **Autograd**: Automatic differentiation; computes gradients
  - **Layers**: Reusable building blocks (Dense, Conv2D, LSTM, etc.)
  - **Model**: Combination of layers with a training loop (compile, fit, evaluate)

PyTorch

- **Framework**: Open-source by Meta; researcher-friendly
- **Dynamic computation graphs**: More flexible than TensorFlow
- **Similar concepts**: Tensors, autograd, modules (layers)
- **Popular for**: Research, rapid prototyping

Key Analogies for Memory & Exams

Concept	Analogy
Encoder–Decoder	Encoder = listener storing meaning; Decoder = speaker expressing in another language
Self-Attention	All students can see what everyone says at once (vs RNN = one-by-one sequentially)
Masked Attention	Student writing essay cannot look at future words they haven't written yet
Pre-training + Fine-tuning	Read thousands of books (pre-training) → light coaching for specific exam (fine-tuning)
RLHF	Student reads books → teacher scores answers → student adjusts to score higher
Tokenization	Cut paragraph into individual word tiles
Stemming	Rough scissors trim (may create non-words)
Lemmatization	Precise cutting machine that respects grammar
Hallucination	Extremely well-read parrot-philosopher: fluent but confidently wrong without guardrails
Safety as multi-layer	Clean ingredients (data) + train chef (alignment) + restaurant inspector (filters + audits)
Deployment compression	Shrink machine (quantization), share among users (batching), or use lighter machine (right-sizing)

Quick MCQ Answer Patterns

Transformers & NLP

- **Transformers** > **RNNs**: Long-range dependencies, parallelism, faster
- **BERT** = **encoder-only**, MLM + NSP, bidirectional, fine-tune for tasks
- **Attention** = **soft weighting** over input elements; solves bottleneck problem
- **Masked attention**: Decoder cannot see future tokens

LLMs & Safety

- **LLM capabilities**: NLP tasks (generation, translation, QA, coding)
- **LLM limitations**: Hallucination, bias, lack of grounding, prompt sensitivity
- **Alignment ≠ perfect**: Adversarial prompts can still trigger harm
- **RLHF pipeline**: Pre-train → SFT → Collect preferences → Train RM → RLHF
- **Bias mitigation**: Data filtering, debiased RM, safety filters, red-teaming
- **Defense-in-depth**: Multiple layers needed; alignment alone insufficient

Text & Representation

- **Preprocessing**: Tokenization (split), Stemming (rough), Lemmatization (accurate)
- **BoW**: Word counts, ignores order
- **TF-IDF**: Weighted counts (common words low weight)
- **Embeddings**: Dense vectors, similar words similar vectors
- **Contextual**: Word meaning depends on context (BERT)

Deployment

- **Infrastructure**: GPUs/TPUs mandatory for reasonable latency
- **Challenges**: High compute cost, memory bottleneck, variable traffic
- **Efficiency**: Quantization, pruning, distillation, batching, caching
- **Monitoring**: Latency, throughput, errors, safety incidents

Neural Networks & Optimization

- **Single perceptron**: Linearly separable problems only (XOR limitation)
- **Multi-layer**: Learn non-linear patterns with hidden layers
- **Activation**: ReLU (modern), Sigmoid/Tanh (historical)
- **Regularization**: Early stopping, dropout, L1/L2
- **Optimizers**: Adam (popular), Momentum (escapes plateaus), SGD (classic)
- **Backprop**: Chain rule for gradients; enables deep learning

---

Exam-Ready Checklist

✓ **Transformers**: Self-attention, parallel, long-range, > RNNs
✓ **BERT**: Bidirectional encoder, MLM + NSP, pre-train once fine-tune many
✓ **Attention**: Soft weighting, relevance scores, solves bottleneck
✓ **Masked attention**: Decoder cannot peek future (auto-regressive)
✓ **LLM capabilities**: Generation, translation, QA, coding, summarization
✓ **LLM limits**: Hallucination, bias, no grounding, prompt sensitive, alignment limits
✓ **RLHF**: Pre-train → SFT → RM training → Reward-guided updates

✓ **Alignment ≠ perfect safety**: Multi-layer defense (data + model + system + monitoring)
✓ **Tokenization > Stemming > Lemmatization**: More complex & accurate
✓ **BoW < TF-IDF < Embeddings < Contextual**: More sophisticated
✓ **Deployment**: GPUs, quantization/distillation, batching, monitoring
✓ **Regularization**: Early stopping, dropout, L1/L2 prevent overfitting
✓ **Adam optimizer**: Adaptive learning rate, very popular
✓ **Autoencoders**: Compress–reconstruct, unsupervised feature learning
✓ **Pre-train once, fine-tune many**: BERT & LLM transfer learning paradigm

---

Document Status: Complete Deep Learning Module (Sessions 17–30)
Last Updated: 2026-01-15
Format: MCQ-optimized with analogies, tables, key patterns
Ready for: CCEE (CDAC) exam revision